Parallel CSR Sparse Matrix–Vector Product for Graphs with 10^8 Rows

Gustavo Lozano

November 2026

Abstract

This document turns the analysis of the notes *Matrix–Vector Multiplication $y = Ax$: Dense versus CSR Storage* [1] into a working OpenMP implementation for the largest matrices of the SuiteSparse collection, such as the MAWI traffic graphs ($1.3 \cdot 10^8$ rows, $2.7 \cdot 10^8$ nonzeros). It explains what changes at that scale (64-bit indices, memory budget, file conversion), presents four parallel kernels that differ only in how they split the work (static rows, dynamic rows, nnz-balanced rows, and merge-path), and reports correctness tests and first measurements. The target machine is a laptop with 32 GB of memory.

Contents

1 Goal and target matrices **1**

2 What changes at this scale **2**
2.1 Index widths . . . 2
2.2 Memory budget . . . 2
2.3 Reading the input . . . 2

3 The four kernels **3**
3.1 Merge-path SpMV . . . 3
3.2 Code . . . 3
3.3 Memory placement . . . 6

4 Correctness **6**

5 First measurements **6**

6 Running on mawi_201512020130 **7**

7 Possible improvements **8**

1 Goal and target matrices

We compute $y = Ax$ with $A \in \mathbb{R}^{M \times N}$ stored in CSR (notes, Sec. 2.2), for matrices that fit in 32 GB. The first target is `mawi_201512020130` from the SuiteSparse collection [3]: an undirected weighted graph built from internet backbone traffic traces, with 128 568 730 rows and columns, 270 234 840 nonzeros, symmetric, with integer values.

Graphs of this kind have about two nonzeros per row on average, but their degree distribution is very uneven: most rows are short and a few rows can be extremely long. That is the situation where the choice of parallel work split matters most (Sec. 3).

2 What changes at this scale

2.1 Index widths

The notes use 32-bit indices throughout. A signed 32-bit integer stops at $2^{31}-1 \approx 2.1\cdot10^9$. Several SuiteSparse matrices have more nonzeros than that (GAP-urand and GAP-kron have 4.2–$4.3\cdot10^9$, AGATHA_2015 has $1.16\cdot10^{10}$), so positions into `val` and `col` need 64 bits. Row and column numbers stay below 2^{31} for every matrix considered, so:

$$\texttt{row_ptr} : \texttt{int64_t}, \qquad \texttt{col} : \texttt{int32_t}, \qquad \texttt{val} : \texttt{double},$$

$$\text{memory}_{\text{CSR}} = 12\,\text{nnz} + 8(M+1) \text{ bytes}, \qquad \text{memory}_{x,y} = 8N + 8M \text{ bytes}.$$

Making `col` 64-bit as well would cost 4 more bytes per nonzero, about 33% more memory *and* memory traffic, for no benefit.

2.2 Memory budget

Matrix	rows M	nnz	CSR + x + y
Spielman_k600	$72.2\cdot10^6$	$217\cdot10^6$	4.3 GB
mawi_201512020130	$128.6\cdot10^6$	$270\cdot10^6$	6.3 GB
kmer_P1a	$139.4\cdot10^6$	$298\cdot10^6$	6.9 GB
mawi_201512020330	$226.2\cdot10^6$	$480\cdot10^6$	11.2 GB
webbase-2001	$118.1\cdot10^6$	$1.02\cdot10^9$	15.1 GB
GAP-kron, GAP-urand	$134.2\cdot10^6$	4.2–$4.3\cdot10^9$	54–55 GB
AGATHA_2015	$184.0\cdot10^6$	$11.6\cdot10^9$	144 GB

On a 32 GB laptop everything up to webbase-2001 fits, with room for the operating system. The GAP and AGATHA matrices need a larger machine.

2.3 Reading the input

Matrix Market files [4] are text: one line `i j value` per stored entry, 1-based. For symmetric matrices only one triangle is stored, so each off-diagonal entry a_{ij} also stands for a_{ji}. Parsing 10^8–10^9 text lines takes minutes, far longer than one product, so the work is split in two programs:

1. `mtx2csr` converts the `.mtx` file to a binary CSR file, once;
2. `spmv_bench` loads the binary file (seconds) and runs the kernels.

Conversion without a temporary copy. The usual approach reads all entries into a coordinate (COO) list and then sorts it, which needs about 16 extra bytes per nonzero. The converter instead reads the file *twice*:

1. **Count:** for each entry add 1 to the length of row i (and of row j if the file is symmetric and $i \neq j$). A prefix sum turns the lengths into `row_ptr`.
2. **Fill:** for each entry, put (j, a_{ij}) at the next free slot of row i (and (i, a_{ij}) in row j if symmetric). `row_ptr[i]` itself serves as the "next free slot" counter, and is shifted back by one position at the end.

Peak memory is exactly the final CSR. Files of type `pattern` (graphs without weights) get the value 1.0 for every entry.

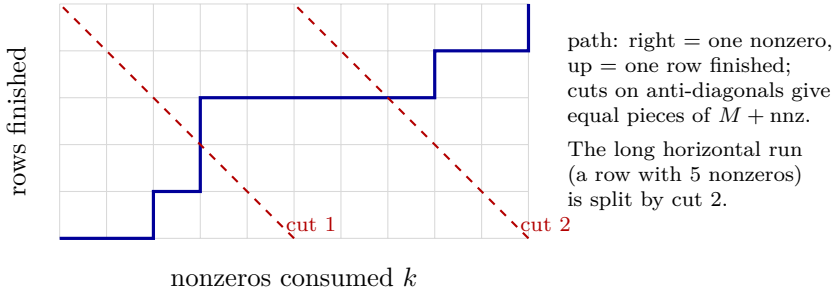
3 The four kernels

All kernels perform the computation of Algorithm 3 of the notes; each y_i is written by exactly one thread, except in the merge-path kernel where a row may be shared (see below). They differ only in how the work is split among the T threads.

Kernel	Split	Weakness
`static`	M/T consecutive rows per thread (`schedule(static)`)	a thread that owns a long row does much more work
`dynamic`	chunks of 4096 rows on demand (`schedule(dynamic,4096)`)	scheduling overhead; one chunk with a huge row is still one chunk
`nnz`	consecutive rows holding about nnz/T nonzeros (notes, Alg. 5)	a single row with more than nnz/T nonzeros cannot be split
`merge`	equal shares of rows + nonzeros; rows may be split [2]	small fix-up step after the parallel region

3.1 Merge-path SpMV

Merrill and Garland [2] view the product as a path of $M + \text{nnz}$ steps: one step for each nonzero (a multiply–add) and one step for each row end (a store of y_i). Cutting this path into T equal pieces balances both kinds of work at once: many short rows (where the per-row cost dominates) and a few long rows (where the per-nonzero cost dominates).



Each thread finds the start and end of its piece by a binary search along an anti-diagonal ($O(\log M)$). A piece can begin or end inside a row. The thread then:

- stores y_i for every row that *ends* inside its piece (for the first row this may be only the tail of the row);
- keeps the partial sum of the row that is still open at the end of its piece as a *carry* (`carry_row[t]`, `carry_val[t]`).

After the implicit barrier of the parallel region, one thread adds the T carries to y. A row longer than several pieces is simply split among several threads, each contributing a carry. No locks or atomics are needed: during the parallel phase every y_i is written by one thread only, and the carries are written to separate array slots.

3.2 Code

`spmv_kernels.c`

```
/*
 * spmv_kernels.c -- parallel CSR sparse matrix-vector product y = A x.
 *
 * All kernels compute the same thing (Algorithm 3 of the notes
 * "Matrix-Vector Multiplication y = Ax: Dense versus CSR Storage").
 * They differ only in HOW the work is split among threads:
```

```
 *
 *   static  : equal number of ROWS per thread            (Lesson 4)
 *   dynamic : chunks of rows handed out on demand        (Lesson 5)
 *   nnz     : equal number of NONZEROS per thread        (notes, Alg. 5)
 *   merge   : equal number of ROWS + NONZEROS per thread,
 *             long rows may be split between threads     (Merrill & Garland 2016)
 */
#include <stdlib.h>
#include <omp.h>
#include "spmv_kernels.h"

#define DYNAMIC_CHUNK 4096   /* rows per chunk; 1 would be far too fine for 10^8 rows */

/* ---------------------------------------------------------------- */
void spmv_serial(const csr_t *A, const double *x, double *y)
{
    const int64_t *rp = A->row_ptr; const int32_t *ci = A->col; const double *v = A->val;
    for (int64_t i = 0; i < A->M; i++) {
        double s = 0.0;
        for (int64_t k = rp[i]; k < rp[i + 1]; k++)
            s += v[k] * x[ci[k]];          /* indirect access (gather) to x */
        y[i] = s;
    }
}

/* ---------------------------------------------------------------- */
/* Rows are independent: each y[i] is written by exactly one iteration,
   so no synchronization is needed (notes, Alg. 4).                  */
void spmv_static(const csr_t *A, const double *x, double *y)
{
    const int64_t *rp = A->row_ptr; const int32_t *ci = A->col; const double *v = A->val;
    #pragma omp parallel for schedule(static)
    for (int64_t i = 0; i < A->M; i++) {
        double s = 0.0;
        for (int64_t k = rp[i]; k < rp[i + 1]; k++)
            s += v[k] * x[ci[k]];
        y[i] = s;
    }
}

void spmv_dynamic(const csr_t *A, const double *x, double *y)
{
    const int64_t *rp = A->row_ptr; const int32_t *ci = A->col; const double *v = A->val;
    #pragma omp parallel for schedule(dynamic, DYNAMIC_CHUNK)
    for (int64_t i = 0; i < A->M; i++) {
        double s = 0.0;
        for (int64_t k = rp[i]; k < rp[i + 1]; k++)
            s += v[k] * x[ci[k]];
        y[i] = s;
    }
}

/* ---------------------------------------------------------------- */
/* Smallest i in [0, M] with row_ptr[i] >= target (binary search).     */
static int64_t lower_bound(const int64_t *rp, int64_t M, int64_t target)
{
    int64_t lo = 0, hi = M;
    while (lo < hi) {
        int64_t mid = lo + (hi - lo) / 2;
        if (rp[mid] < target) lo = mid + 1; else hi = mid;
    }
    return lo;
}
```

```c
/* nnz-balanced partition (notes, Alg. 5): thread t gets the contiguous
   rows start(t) .. start(t+1)-1, holding about nnz/T nonzeros. Each thread
   computes its own boundaries, so there is no shared partition array.
   Weakness: a single row longer than nnz/T cannot be split.            */
void spmv_nnz(const csr_t *A, const double *x, double *y)
{
    const int64_t *rp = A->row_ptr; const int32_t *ci = A->col; const double *v = A->val;
    #pragma omp parallel
    {
        int T = omp_get_num_threads(), t = omp_get_thread_num();
        int64_t r0 = (t == 0)     ? 0    : lower_bound(rp, A->M, A->nnz * t / T);
        int64_t r1 = (t == T - 1) ? A->M : lower_bound(rp, A->M, A->nnz * (t + 1) / T);
        for (int64_t i = r0; i < r1; i++) {
            double s = 0.0;
            for (int64_t k = rp[i]; k < rp[i + 1]; k++)
                s += v[k] * x[ci[k]];
            y[i] = s;
        }
    }
}

/* ---------------------------------------------------------------------- */
/* Merge-path SpMV (Merrill & Garland, SC16).
 *
 * Think of the work as a path of M + nnz steps: one step per nonzero
 * (multiply-add) and one step per row end (store y[i]). Cutting this path
 * into T equal pieces balances BOTH kinds of work: many short rows
 * (per-row overhead) and a few huge rows (per-nonzero work). A piece may
 * start or end in the middle of a row; the partial sum of an unfinished
 * row is kept as a "carry" and added after the parallel region.        */

/* Find where diagonal d of the merge path crosses: returns the number of
   finished rows (*row) and of consumed nonzeros (*k), with *row + *k = d. */
static void merge_search(const int64_t *rp, int64_t M, int64_t nnz,
                         int64_t d, int64_t *row, int64_t *k)
{
    int64_t lo = (d > nnz) ? d - nnz : 0;
    int64_t hi = (d < M)   ? d       : M;
    while (lo < hi) {
        int64_t mid = lo + (hi - lo) / 2;
        if (rp[mid + 1] <= d - mid - 1) lo = mid + 1;  /* row mid ends before this point */
        else                            hi = mid;
    }
    *row = lo;
    *k   = d - lo;
}

void spmv_merge(const csr_t *A, const double *x, double *y)
{
    const int64_t *rp = A->row_ptr; const int32_t *ci = A->col; const double *v = A->val;
    int T = omp_get_max_threads();
    int64_t *carry_row = malloc((size_t)T * sizeof(int64_t));
    double  *carry_val = malloc((size_t)T * sizeof(double));
    int64_t total = A->M + A->nnz;

    #pragma omp parallel num_threads(T)
    {
        int t = omp_get_thread_num();
        int64_t row, k, row_end, k_end;
        merge_search(rp, A->M, A->nnz, total * t / T,       &row,     &k);
        merge_search(rp, A->M, A->nnz, total * (t + 1) / T, &row_end, &k_end);

        /* rows that END inside my piece: compute and store */
        double s = 0.0;
```

```
        for (; row < row_end; row++) {
            for (; k < rp[row + 1]; k++)
                s += v[k] * x[ci[k]];
            y[row] = s;             /* may be only the tail of the row:  */
            s = 0.0;                /* the earlier part arrives as a carry */
        }
        /* the row that is still open at the end of my piece */
        for (; k < k_end; k++)
            s += v[k] * x[ci[k]];
        carry_row[t] = row_end;
        carry_val[t] = s;
    }   /* implicit barrier: every y[row] above is written */

    /* fix-up: add the partial sums of rows split between threads (O(T)) */
    for (int t = 0; t < T; t++)
        if (carry_row[t] < A->M)
            y[carry_row[t]] += carry_val[t];

    free(carry_row);
    free(carry_val);
}
```

3.3 Memory placement

`csr_read` and `spmv_bench` initialize every large array in parallel with `schedule(static)` before filling it (*first touch*). On a multi-socket server this places each part of the matrix in the memory attached to the core that will use it; on a laptop with one memory controller it has no effect but costs nothing. Threads should be pinned with `OMP_PROC_BIND=close` and `OMP_PLACES=cores` [6].

4 Correctness

Every run of `spmv_bench` computes y with a serial reference first and compares each kernel against it (`max error` column, relative to $\max_i |y_i|$). The tests performed:

1. The 6×6 running example of the notes (Sec. 2.3), stored in `example.mtx`, with $x = (1, \ldots, 6)$: all kernels give $y = (20, 6, 0, 27, 4, 56)$, including the empty row.

2. The converter, on a symmetric synthetic graph with 3000 vertices, against SciPy's Matrix Market reader: products agree to a relative error of $2 \cdot 10^{-15}$.

3. All four kernels on four matrices (with empty rows, and with rows holding up to 12% of all nonzeros) at 1, 2, 3, 5, 7, 16 and 33 threads: 112 runs, all with error at most 10^{-12}, most exactly 0. The odd and large thread counts force the merge-path kernel to split rows between several threads.

Results may differ from the serial reference in the last bits because the merge kernel adds the pieces of a split row in a different order (notes, Sec. 9).

5 First measurements

The development machine (2 cores of an Intel Xeon, 7 GB of memory) cannot hold the MAWI matrix, so the measurements use a synthetic graph of similar type (about two nonzeros per row, a few very long rows), made by `gen_graph`: $M = 2 \cdot 10^7$, nnz $= 4.2 \cdot 10^7$ (2.1 per row), 32% empty rows, and 4 hub rows that together hold 45% of the nonzeros (the largest one 11%).

Partition quality. `spmv_bench` prints the work of the busiest thread divided by the average, with work = rows + nonzeros. It depends only on the matrix and T, not on the machine:

	$T = 2$	$T = 4$	$T = 8$	$T = 16$
synthetic, hubs 3% each: static	1.08	1.25	1.59	2.27
synthetic, hubs 3% each: nnz	1.05	1.05	1.05	1.05
synthetic, hubs 11% each: static	1.30	–	3.13	–
synthetic, hubs 11% each: nnz	1.26	–	1.26	–
merge (any matrix)	1.00	1.00	1.00	1.00

With 8 threads the static split gives one thread more than three times the average work, so it cannot run faster than about $8/3.13 \approx 2.6$ times the serial speed.

Times. Hub graph (11% per hub), best of 10 runs:

kernel	1 thread (s)	2 threads (s)	speed-up	GB/s (2 thr.)
serial	0.490	–	–	–
static	–	0.310	1.58	3.17
dynamic	–	0.248	1.98	3.97
nnz	–	0.274	1.79	3.60
merge	–	0.283	1.73	3.48

Two observations:

- With only 2 threads the partitions do not differ much; the static penalty grows with T (table above). The real comparison is on a machine with more cores.
- The achieved bandwidth (3–4 GB/s) is far below the machine's memory bandwidth. The `GB/s` column uses the *best-case* traffic of the notes (Sec. 4.2), which assumes x stays in cache. Here x has $2 \cdot 10^7$ entries (160 MB) and is read at random positions, so almost every access to x misses the cache and loads a whole 64-byte line for 8 useful bytes: the *worst case* of the notes, made worse by the cache line. On this kind of graph the gather from x, not the matrix, dominates the cost.

Equal *work* is also not exactly equal *time*: a nonzero whose x_j is in cache costs much less than one that misses. The hub rows read x at random positions, while the short rows mostly read x_{hub}, which stays in cache. This is why the dynamic schedule, which reacts to the real time spent, can beat the merge split on a given machine.

6 Running on mawi_201512020130

```
make && make test
wget https://suitesparse-collection-website.herokuapp.com/MM/MAWI/mawi_201512020130.tar.gz
tar xzf mawi_201512020130.tar.gz
./mtx2csr mawi_201512020130/mawi_201512020130.mtx mawi_130.csr
export OMP_PROC_BIND=close OMP_PLACES=cores
for t in 1 2 4 $(nproc); do OMP_NUM_THREADS=$t ./spmv_bench mawi_130.csr 10; done
```

What to expect. One product moves at least $12 \cdot 2.70 \cdot 10^8 + 8 \cdot 1.29 \cdot 10^8 \cdot 3 \approx 6.3$ GB. At a laptop memory bandwidth of 20–40 GB/s the roofline model (notes, Sec. 5) gives a lower bound of 0.16–0.32 s per product; the random gather from x (1 GB, far larger than the cache) will add to that. The things to look at in the output are:

1. the `rows` line: the length of the longest row and its share of nnz;
2. the `imbalance` line for your number of cores;
3. how close each kernel gets to the bandwidth bound, and which split wins.

7 Possible improvements

- **Locality of x.** Reordering rows and columns (e.g. reverse Cuthill–McKee, or ordering vertices by degree) so that nearby rows read nearby entries of x attacks the dominant cost found above (notes, Sec. 7).
- **Pattern matrices.** For graphs without weights (kmer, GAP-kron), drop `val` and add x_j directly: 4 instead of 12 bytes per nonzero.
- **SIMD.** `#pragma omp simd reduction(+:s)` on the inner loop; limited by the gather and the short rows (notes, Sec. 8.3; van der Pas et al. [6]).
- **Validation against a library**, such as Intel MKL's `mkl_sparse_d_mv`, for both correctness and speed (notes, Sec. 10).

Key ideas

- At 10^8 rows, index width, memory budget and file conversion are design decisions in their own right.
- The kernels differ only in the work split. The merge path balances rows and nonzeros exactly and splits long rows, with a carry fix-up instead of locks.
- For irregular graphs the random gather from x dominates: the measured bandwidth is far below the best-case model, so data locality matters more than the arithmetic.

References

[1] G. Lozano. *Matrix–Vector Multiplication $y = Ax$: Dense versus CSR Storage*. Notes, November 2026.

[2] D. Merrill and M. Garland. Merge-based parallel sparse matrix–vector multiplication. In *Proceedings of SC16*, 2016.

[3] T. A. Davis and Y. Hu. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1–25, 2011. https://sparse.tamu.edu

[4] R. F. Boisvert, R. Pozo, and K. A. Remington. The Matrix Market exchange formats: initial design. NISTIR 5935, NIST, 1996.

[5] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.

[6] R. van der Pas, E. Stotzer, and C. Terboven. *Using OpenMP — The Next Step: Affinity, Accelerators, Tasking, and SIMD*. MIT Press, Cambridge, MA, 2017.